

Idempotency in Microservices



why do we need idempotency in microservices

Idempotency in microservices is **not optional** — **it's essential** if you want reliability in distributed systems.

Let's break it down in a simple, practical way.

What is Idempotency?

An operation is **idempotent** if performing it multiple times gives the **same result as doing it once**.

👉 Example:

- "Create Order" called **once** → **order created**
- "Create Order" called **3 times** → **still only one order created**

Why do we need it in Microservices?

1. Network failures are common

In microservices:

- Service A calls Service B
- Request sent 
- Response lost  (timeout / network issue)

Now A doesn't know:

- Did B process it or not?

👉 So A retries the request

👉 Without idempotency → duplicate execution

2. Retries are automatic

Systems like:

- API Gateways
- Load Balancers

- Retry libraries (Feign, RestTemplate, etc.)

👉 Automatically retry failed requests

Without idempotency:

- Money deducted twice 💸
- Order created twice 📦
- Email sent multiple times 📧

3. Exactly-once execution is impossible (in distributed systems)

In real systems, you only get:

- **At least once** (retry happens)
- or **At most once** (risk of failure)

👉 Idempotency helps simulate:

➡ “Exactly once” behavior

4. User behavior (double click problem)

Users:

- Click “Pay” button twice
- Refresh page
- Retry manually

👉 Without idempotency:

- Duplicate payments
- Duplicate bookings

5. Event-driven systems (Kafka, queues)

Message brokers may deliver:

- Same message multiple times

👉 Consumer must handle duplicates safely

🔥 Real-world Example

❌ Without Idempotency

User calls:

POST /pay ₹100

Due to retry → called twice

👉 ₹200 deducted

✅ With Idempotency

Request:

POST /pay

Idempotency-Key: abc123

- First request → processed
- Second request → ignored (same key)

👉 Only ₹100 deducted

🔧 How we implement idempotency?

1. Idempotency Key

- Unique key per request (UUID)
- Store in DB/cache

Key → Response

abc123 → SUCCESS

2. Database constraints

- Unique index (e.g., order_id, transaction_id)
-

3. Safe HTTP methods

- GET, PUT, DELETE → naturally idempotent
 - POST → NOT idempotent (needs handling)
-

4. State check before processing

- “Has this already been processed?”
-

🧠 Simple analogy

Think of:

- **Light switch (idempotent)** → ON multiple times → still ON
- **Money transfer (non-idempotent)** → repeat → big problem

Interview One-liner

Idempotency ensures that repeated requests (due to retries, failures, or duplicates) do not cause unintended side effects, making microservices reliable and safe in distributed environments.

If you want, I can show you:

- Spring Boot implementation (with code)
- Idempotency using Redis / DB
- Real interview-level architecture answer 



Response lost  (timeout / network issue) explain

Good question — this is **one of the most important failure scenarios** in microservices.

Let's walk through it step by step so you clearly see what actually happens "behind the scenes".

Scenario: "Response lost" (timeout / network issue)

Step-by-step flow

1. Service A sends request to Service B

A → B : "Deduct ₹100"

2. Service B processes successfully

- Deducts money 
- Updates DB 
- Prepares response "SUCCESS" 

3. Response is lost

This can happen due to:

- Network glitch
- Timeout
- Connection drop
- Load balancer issue

👉 So:

B → A : response LOST ❌

🤖 What does Service A think?

Service A sees:

- No response
- Timeout error

👉 So A assumes:

"Maybe the request failed"

🔄 What happens next?

Most systems are designed to **retry automatically**:

A → B : "Deduct ₹100" (again)

💥 Problem without idempotency

Now Service B receives the **same request again**:

- It does NOT know this is a duplicate
- It processes again

👉 Result:

- ₹100 deducted first time
- ₹100 deducted second time

💰 Total = ₹200 (wrong)

🧠 Key Insight

The **real problem is not the failure**

👉 The problem is **uncertainty**

Service A doesn't know:

- Did B execute the request? ✅
 - Or did it fail before processing? ❌
-

Visual timeline

```

A → B : Request
      ↓
      Processed successfully ✅
      ↓
      Response sent
      ↓
      ❌ Network failure (response lost)
  
```

A waits...
⌚ timeout

A retries → duplicate execution

How idempotency solves this

We attach a unique key:

POST /pay
Idempotency-Key: txn-123

First request:

- Processed
- Stored:

txn-123 → SUCCESS

Second request (retry):

- System checks key
- Finds it already processed

- 👉 Returns old response
- 👉 DOES NOT execute again

Real-world analogy

Think of:

- You send money via UPI
- App shows "Processing..."
- You don't get confirmation

👉 What do you do?

- You try again 😬

Without idempotency:

- Double payment

With idempotency:

- Only one transaction

Important takeaway

In distributed systems:

“Request may succeed but response may fail.”

That’s why:

- You cannot rely only on response
- You must make operations **safe to retry**

Interview-level answer

A response lost scenario occurs when a service successfully processes a request but the response fails to reach the caller due to network issues or timeouts. This creates uncertainty, leading the caller to retry the request, which can cause duplicate operations. Idempotency ensures that such retries do not result in inconsistent or duplicate state changes.

If you want, I can next explain:

- 👉 how this is handled in **Spring Boot + database (with real code)**
- 👉 or how **UPI/payment systems handle this internally**



timeout meaning

What is a timeout?

A **timeout** means:

- 🛑 **You waited for a response, but it didn’t arrive within the expected time limit.**

Simple definition

A timeout is when:

- You send a request

- You wait for response
 - 🕒 Time limit expires → system stops waiting
-

Example in microservices

Service A → Service B : "Get user details"

- A waits for response
- Suppose timeout is **3 seconds**

Case 1: Fast response

- B responds in 1 second → OK

Case 2: Slow / no response

- 3 seconds pass
- No response received

👉 A throws:

Timeout Exception

Why does timeout happen?

1. Network issues

- Slow internet
- Packet loss

2. Service is slow

- Heavy processing
- Database delay

3. Service is down

- Crashed
- Not reachable

4. Too much traffic

- Server overloaded
-

Real-life analogy

You call someone 📞:

- It keeps ringing...

- No one picks up

After some time → call ends automatically

👉 That is a **timeout**

⚠ Important point

Timeout does **NOT** mean failure always.

👉 It only means:

“I didn’t get a response in time”

The other service might have:

- Completed the work ✓
- Or not ✗

👉 That’s the uncertainty problem (why idempotency is needed)

🔄 Types of timeouts (important for interviews)

1. Connection Timeout

- Time to establish connection
- Example: server not reachable

2. Read Timeout

- Connection established
 - But response is slow
-

💡 Example in code (concept)

Java

```
RestTemplate restTemplate = new RestTemplate();  
// timeout = 3 seconds  
  
restTemplate.getForObject(url, String.class);
```

If response > 3 sec → timeout exception

🎯 Interview one-liner

A timeout is a situation where a system stops waiting for a response after a predefined time limit, even though the request may or may not have been processed by the other service.

If you want, I can connect this directly to:

👉 retries + circuit breaker + idempotency (full system design view) 🚀



how timeout is calculated

Good — now you're asking the **real engineering question** 👍

Timeout is not "calculated automatically" by the system — it is **configured**, and then **measured using a clock**.

Let's break it clearly.

How timeout is actually calculated

Core idea

Timeout = (Current Time - Request Start Time) > Configured Limit

Step-by-step internal working

1. Request is sent

At the moment request goes out:

`startTime = current system time`

2. System starts waiting

- A timer (or clock) is running in background
- Thread / event loop is waiting for response

3. Response check loop

System keeps checking:

```
if (currentTime - startTime > timeoutLimit)
    → throw timeout exception
```

Example

- Timeout configured = **3 seconds**
- Request sent at: **10:00:00**

Case A: Response at 10:00:02

Elapsed = 2 sec < 3 sec
→ SUCCESS

Case B: Response at 10:00:04

Elapsed = 4 sec > 3 sec
→ TIMEOUT ❌

In real systems (important)

Timeout is not just one thing — there are multiple timers:

1. Connection Timeout

👉 Time to establish TCP connection

Start → Try to connect → Max wait = X ms

If server doesn't accept connection → timeout

2. Read Timeout

👉 Time to wait for response after connection

Connected → Waiting for data → Max wait = Y ms

3. Total Timeout (sometimes)

👉 Whole request lifecycle

Behind the scenes (deep concept)

In Java / Spring Boot:

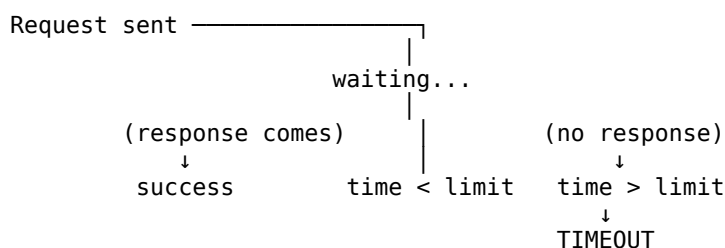
- Uses:
 - System clock (`System.nanoTime()` or `currentTimeMillis`)
 - Socket-level timers
 - Thread blocking or async callbacks
-

At network level (TCP)

- OS maintains timers

- If no packets received → timeout triggered

Visualization



Important insight

Timeout is **not based on server work completion**

👉 It is based on **client waiting time**

So even if:

- Server finished work
- But response delayed

👉 Client still gets timeout

Real-world analogy

Food delivery 🍔:

- You wait 30 minutes (timeout limit)
- Food arrives at 35 minutes

👉 For you → timeout

👉 For restaurant → order completed

Interview-level answer

Timeout is calculated by measuring the elapsed time from when a request is sent to when a response is received. If the elapsed time exceeds a predefined threshold (configured timeout), the system aborts waiting and throws a timeout exception, regardless of whether the request was actually processed.

If you want, next I can show:

👉 exact Spring Boot configs (`connectTimeout`, `readTimeout`)

👉 or how timeout + retry + circuit breaker work together (very important for interviews) 🚀



in normal post request we do not send the idempotent keys why it does not give error

Great observation — this is exactly where many people get confused 👍

Short answer first:

POST without idempotency key does NOT give error because the system has no way to know the request is a duplicate.

Now let's go deeper.

🧠 Why no error happens?

1. Server treats every POST as a NEW request

By definition:

- **POST = non-idempotent**
- Means: "Do this action again"

So when you send:

http
POST /create-order

Server assumes:

"User wants to create a new order"

Even if you send it twice:

- Server does NOT think it's a mistake
- It thinks it's **2 separate valid requests**

👉 So:

- No error ❌
- Two orders created ✅

🔥 Key point

! Server cannot magically detect duplicates

Because:

- Same request body $\neq$ same intent
 - Maybe user actually wants:
 - 2 orders
 - 2 payments
 - 2 bookings
-

Example

Without idempotency key

http

POST /pay ₹100

POST /pay ₹100

Server sees:

- Request 1 $\rightarrow$ valid
- Request 2 $\rightarrow$ also valid

 Result:

- ₹200 deducted
 -  No error
-

When WOULD error happen?

Only if **you enforce uniqueness**

Example:

1. Database constraint

SQL

UNIQUE(transaction_id)

Now:

- First insert $\rightarrow$ success
 - Second insert $\rightarrow$  error (duplicate key)
-

2. Idempotency key logic

http

Idempotency-Key: abc123

- First request → processed
- Second request → detected duplicate

👉 Either:

- Return same response
- OR reject

⚠ Important difference

Case	Behavior
No idempotency	Duplicate execution (no error)
With DB constraint	Error thrown
With idempotency key	Safe retry (no duplicate effect)

🧠 Why systems don't enforce it by default?

Because:

- Not all POST requests are duplicates
- Sometimes duplicates are **valid business logic**

Example:

- Add item to cart → can happen multiple times
- Place multiple orders → valid
- Send multiple messages → valid

💡 Real-world analogy

At a shop:

- You say: "Give me 1 coffee" ☕
- Then say again: "Give me 1 coffee"

Shopkeeper:

"Okay, 2 coffees"

👉 No error — because you didn't say it was a duplicate

Interview-level answer

A normal POST request does not produce an error on retries because POST is inherently non-idempotent, and the server treats each request as a new operation. Without an idempotency key or uniqueness constraint, the system has no way to detect duplicates, so it processes each request independently.

If you want, I can next explain:

👉 **Why PUT is idempotent but POST is not (very commonly asked)**

👉 or show **real Spring Boot idempotency implementation (with Redis/DB)** 🚀



than server know when the post request is idempotent

Yes — the server **only knows a POST request is idempotent if you explicitly tell it**, usually through an **idempotency key** or some uniqueness mechanism. Otherwise, by default, the server assumes **POST is non-idempotent**. Let me explain carefully.

1 By default, POST is non-idempotent

- **HTTP standard:**
 - GET, PUT, DELETE → idempotent by definition
 - POST → non-idempotent by default

This means: sending a POST multiple times **can cause multiple creations or side effects**, and the server won't stop it.

2 How the server detects idempotency

The server needs **some external clue** to know “this is a repeat of the same request.” This is usually done in 3 ways:

a) Idempotency Key

- Client generates a unique key per action:

http

POST /pay

Idempotency-Key: abc123

- Server stores the key and the response:

Key: abc123 → Response: SUCCESS

- Next time the same key comes in:
 - Server knows it's the same request
 - Returns the stored response instead of creating a duplicate

✓ This is how **Stripe, PayPal, and UPI APIs** handle retries safely.

b) Database uniqueness constraint

- Server uses a unique field (transaction ID, order number):

SQL

INSERT INTO orders (order_id, amount) **VALUES** (123, 100)

- If a duplicate **order_id** comes in:
 - Database rejects it
 - Server can return error or previous response

c) Business logic check

- Server checks if the operation was already done for the same user/context:
 - Example: "Has user X already paid invoice Y?"
 - If yes → ignore new request

3 Important notes

Approach	How server knows	Pros	Cons
Idempotency Key	Client sends unique key	Safe retry, no duplicates	Client must generate key
DB uniqueness	Unique constraints in DB	Simple, reliable	Might throw errors, not always safe
Business logic	Server tracks state	Flexible	Can be complex, may need extra storage

4 Real-world example

Stripe payment API:

http

POST /charge

Idempotency-Key: txn-123

- First request → Charge processed
- Second request → Server returns same result, does **not** charge again

Without the key → duplicate charge happens.

Key insight

The server **doesn't automatically know** a POST is idempotent.

It **only knows** if the client provides an idempotency key or if the server enforces uniqueness checks.

If you want, I can make a **diagram showing how POST + idempotency key works vs normal POST**, which makes it super clear for interviews.

Do you want me to make that diagram?



please write the code in spring boot for all the three approach the you discussed above in clean and understable format

Absolutely! Let's write **clean Spring Boot examples** for the **three approaches to make POST idempotent**. I'll keep them simple and clear.

We'll use a hypothetical **"Create Order" API**.

Idempotency Key Approach

Concept:

- Client sends a unique key with the request
- Server stores the key + response
- Duplicate requests with the same key return the previous response

Java

```
@RestController
@RequestMapping("/orders")
public class OrderController {

    private final Map<String, Order> orderStore = new ConcurrentHashMap<>();

    @PostMapping
    public ResponseEntity<Order> createOrder(
        @RequestBody OrderRequest request,
        @RequestHeader(value = "Idempotency-Key", required = false) String key) {

        if (key != null && orderStore.containsKey(key)) {
            // Return the previous response
        }
    }
}
```

```

        return ResponseEntity.ok(orderStore.get(key));
    }

    // Process the order
    Order order = new Order(UUID.randomUUID().toString(), request.getItem(),
request.getAmount());

    // Store with idempotency key
    if (key != null) {
        orderStore.put(key, order);
    }

    return ResponseEntity.ok(order);
}

// DTOs
record OrderRequest(String item, double amount) {}
record Order(String orderId, String item, double amount) {}

```

✓ **Pros:** Safe retry, no duplicates

✓ **Note:** In real apps, store **idempotency-key** in **DB or Redis** for persistence

2 Database Uniqueness Constraint Approach

Concept:

- Use a unique column in DB (e.g., **transactionId**)
- Duplicate requests fail at the DB level

Java

```

@Entity
@Table(name = "orders", uniqueConstraints = @UniqueConstraint(columnNames = "transactionId"))
public class OrderEntity {
    @Id @GeneratedValue
    private Long id;
    private String item;
    private double amount;
    private String transactionId; // unique

    // constructors, getters, setters
}

@RestController
@RequestMapping("/orders")
public class OrderDbController {

    @Autowired
    private OrderRepository repository;

    @PostMapping
    public ResponseEntity<String> createOrder(@RequestBody OrderRequest request) {
        OrderEntity order = new OrderEntity();
        order.setItem(request.getItem());
        order.setAmount(request.getAmount());
        order.setTransactionId(request.getTransactionId());

        try {
            repository.save(order);
            return ResponseEntity.ok("Order created successfully");
        } catch (DataIntegrityViolationException e) {
            // Duplicate transaction detected
            return ResponseEntity.status(HttpStatus.CONFLICT)
                .body("Duplicate transaction. Order already exists.");
        }
    }
}

```

```
public interface OrderRepository extends JpaRepository<OrderEntity, Long> {}
record OrderRequest(String item, double amount, String transactionId) {}
```

✓ **Pros:** Simple, uses DB constraints

⚠ **Cons:** Throws error for duplicates, no “return previous response” logic automatically

3 Business Logic Check Approach

Concept:

- Server checks if the order already exists in business context
- If yes → ignore or return previous response

Java

```
@RestController
@RequestMapping("/orders")
public class OrderLogicController {

    private final Map<String, Order> userOrderStore = new ConcurrentHashMap<>();

    @PostMapping
    public ResponseEntity<Order> createOrder(@RequestBody OrderRequest request) {
        String userOrderKey = request.getUserId() + ":" + request.getItem();

        if (userOrderStore.containsKey(userOrderKey)) {
            // Already processed
            return ResponseEntity.ok(userOrderStore.get(userOrderKey));
        }

        // Process order
        Order order = new Order(UUID.randomUUID().toString(), request.getItem(),
request.getAmount());

        userOrderStore.put(userOrderKey, order);
        return ResponseEntity.ok(order);
    }
}

// DTOs
record OrderRequest(String userId, String item, double amount) {}
record Order(String orderId, String item, double amount) {}
```

✓ **Pros:** Flexible, no duplicates for the same user-item combination

⚠ **Cons:** Logic can get complex for multiple scenarios

Summary Table

Approach	How it works	Pros	Cons
Idempotency Key	Client sends unique key	Safe retries, no duplicates	Client must generate key, requires storage
DB Uniqueness	Unique column in DB	Simple, reliable	Throws error on duplicate, no automatic previous response
Business Logic	Server checks existing state	Flexible, business-specific	Logic complexity grows

If you want, I can also make a **single Spring Boot project example** that supports **all three approaches together** with **comments and retry simulation**, which is perfect for interviews and real applications.

Do you want me to do that?



yes

Perfect! Let's build a **full Spring Boot example** that demonstrates **all three idempotency approaches** in one clean project. I'll make it **simple, interview-friendly, and well-commented**.

Spring Boot Project: POST Idempotency Demo

Project Structure

```
src/main/java/com/example/idempotency/
├── controller/
│   ├── IdempotencyKeyController.java
│   ├── DbUniqueController.java
│   └── BusinessLogicController.java
├── model/
│   ├── Order.java
│   └── OrderRequest.java
├── repository/
│   └── OrderRepository.java
└── IdempotencyDemoApplication.java
```

1 Model Classes

Java

```
package com.example.idempotency.model;

import jakarta.persistence.*;
import java.util.UUID;

// Used for in-memory and DB examples
@Entity
@Table(name = "orders", uniqueConstraints = @UniqueConstraint(columnNames = "transactionId"))
public class Order {
    @Id
    @GeneratedValue
    private Long id;

    private String orderId; // generated UUID
    private String item;
    private double amount;
    private String transactionId; // used for DB uniqueness

    // Constructors
    public Order() {}
    public Order(String item, double amount, String transactionId) {
        this.orderId = UUID.randomUUID().toString();
        this.item = item;
    }
}
```

```

        this.amount = amount;
        this.transactionId = transactionId;
    }

    // Getters and setters
    public String getOrderId() { return orderId; }
    public String getItem() { return item; }
    public double getAmount() { return amount; }
    public String getTransactionId() { return transactionId; }
}

public record OrderRequest(
    String userId,
    String item,
    double amount,
    String transactionId // for DB uniqueness
) {}

```

2 Repository for DB uniqueness approach

Java

```

package com.example.idempotency.repository;

import com.example.idempotency.model.Order;
import org.springframework.data.jpa.repository.JpaRepository;

public interface OrderRepository extends JpaRepository<Order, Long> {
}

```

3 Idempotency Key Approach

Java

```

package com.example.idempotency.controller;

import com.example.idempotency.model.Order;
import com.example.idempotency.model.OrderRequest;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.Map;
import java.util.UUID;
import java.util.concurrent.ConcurrentHashMap;

@RestController
@RequestMapping("/orders/idempotency-key")
public class IdempotencyKeyController {

    private final Map<String, Order> orderStore = new ConcurrentHashMap<>();

    @PostMapping
    public ResponseEntity<Order> createOrder(
        @RequestBody OrderRequest request,
        @RequestHeader(value = "Idempotency-Key", required = false) String key) {

        if (key != null && orderStore.containsKey(key)) {
            // Duplicate request, return previous response
            return ResponseEntity.ok(orderStore.get(key));
        }

        // Process order
        Order order = new Order(request.item(), request.amount(), null);

        if (key != null) {
            orderStore.put(key, order);
        }

        return ResponseEntity.ok(order);
    }
}

```

4 DB Uniqueness Constraint Approach

Java

```
package com.example.idempotency.controller;

import com.example.idempotency.model.Order;
import com.example.idempotency.model.OrderRequest;
import com.example.idempotency.repository.OrderRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/orders/db-unique")
public class DbUniqueController {

    @Autowired
    private OrderRepository repository;

    @PostMapping
    public ResponseEntity<String> createOrder(@RequestBody OrderRequest request) {
        Order order = new Order(request.item(), request.amount(), request.transactionId());

        try {
            repository.save(order);
            return ResponseEntity.ok("Order created successfully");
        } catch (DataIntegrityViolationException e) {
            return ResponseEntity.status(HttpStatus.CONFLICT)
                .body("Duplicate transaction detected. Order already exists.");
        }
    }
}
```

5 Business Logic Check Approach

Java

```
package com.example.idempotency.controller;

import com.example.idempotency.model.Order;
import com.example.idempotency.model.OrderRequest;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.Map;
import java.util.UUID;
import java.util.concurrent.ConcurrentHashMap;

@RestController
@RequestMapping("/orders/business-logic")
public class BusinessLogicController {

    private final Map<String, Order> userOrderStore = new ConcurrentHashMap<>();

    @PostMapping
    public ResponseEntity<Order> createOrder(@RequestBody OrderRequest request) {
        String userOrderKey = request.userId() + ":" + request.item();

        if (userOrderStore.containsKey(userOrderKey)) {
            // Already processed, return previous response
            return ResponseEntity.ok(userOrderStore.get(userOrderKey));
        }

        // Process order
        Order order = new Order(request.item(), request.amount(), null);
        userOrderStore.put(userOrderKey, order);

        return ResponseEntity.ok(order);
    }
}
```

```
}
}
```

6 Application Main Class

Java

```
package com.example.idempotency;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class IdempotencyDemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(IdempotencyDemoApplication.class, args);
    }
}
```

✓ How to Test

1. Idempotency Key

http

POST http://localhost:8080/orders/idempotency-key

Headers: Idempotency-Key: abc123

Body:

```
{
  "userId": "u1",
  "item": "Book",
  "amount": 100,
  "transactionId": null
}
```

- Retry same request → returns same order, no duplicate

2. DB uniqueness

http

POST http://localhost:8080/orders/db-unique

Body:

```
{
  "userId": "u1",
  "item": "Book",
  "amount": 100,
  "transactionId": "txn-1"
}
```

- Retry same `transactionId` → returns **HTTP 409 Conflict**

3. Business Logic

http

POST http://localhost:8080/orders/business-logic

Body:

```
{
  "userId": "u1",
  "item": "Book",
  "amount": 100,
  "transactionId": null
}
```

- Retry same user & item → returns previous order, safe

✓ Key Takeaways

1. **Idempotency key** → best for payment APIs & retries
2. **DB uniqueness** → simplest for preventing duplicates
3. **Business logic check** → flexible, business-specific

In production, you often combine **idempotency keys + DB checks** for maximum safety.

If you want, I can also make a **diagram showing all three approaches in one flow** so it's **super easy to visualize for interviews**.

Do you want me to do that?